Final Report on The Automaton Auditor

Prepared By: Rediet Bekele

Executive Summary

Scope and approach. This audit exercised an automated, graph-based self-auditor that accepts a repository URL and a PDF path, runs parallel *Detectives* to collect evidence, aggregates that evidence, runs three judge personas (Prosecutor, Defense, TechLead) per rubric dimension, and synthesizes deterministic criterion results via a Chief Justice. The pipeline is implemented as a **StateGraph** with explicit fan-out (detectives → judges) and fan-in (aggregator → chief) synchronization points. The run and artifact are attached to a public LangSmith run for traceability.

Outcome. Aggregate score: 2.70 / 5.00 (54%). The pipeline executed end-to-end and produced a full audit_report.json. The system demonstrates the intended architecture and adjudication pattern, but several high-impact gaps reduce forensic confidence and reproducibility.

Top finding and priority. The most impactful shortcoming is **insufficient git provenance**: commit SHAs are present but per-commit metadata (author, timestamp, diffs, signatures) is missing. The primary remediation priority is **state management rigor**, convert state and domain objects to typed models (Pydantic), add state snapshot tests, and validate serialization. Addressing state rigor will reduce brittle detector outputs and prevent serialization/upload failures.

Immediate actions for an engineer. (1) Add Pydantic AgentState and domain models and snapshot tests; (2) extend the git detector to collect per-commit metadata and emit evidence IDs;

(3) instrument retry and fallback spans in the retry decorator and re-run to capture error-recovery traces.

Architecture Deep Dive

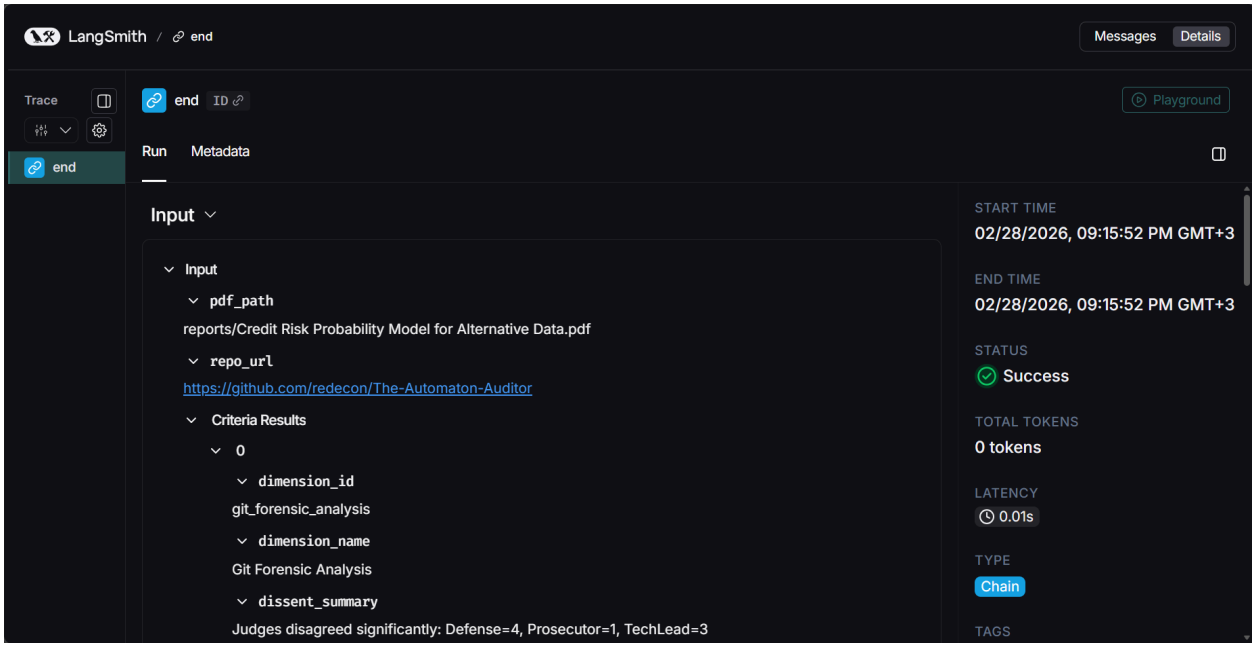

Figure 1.trace_spans
LangSmith trace screenshot showing detectives → aggregator → judges → chief → end.

Dialectical Synthesis (concrete mapping). The system implements dialectical synthesis as three judge personas producing independent JudicialOpinion objects per dimension. The Chief Justice applies deterministic aggregation rules (median/MinMax reconciliation, dissent detection, remediation generation) to produce CriterionResult objects. These components map to concrete code locations: judge implementations in src/nodes/judges.py, graph wiring and normalization in src/graph.py, and synthesis logic in src/nodes/justice.py. The deterministic approach yields auditable, reproducible verdicts rather than opaque averaged outputs.

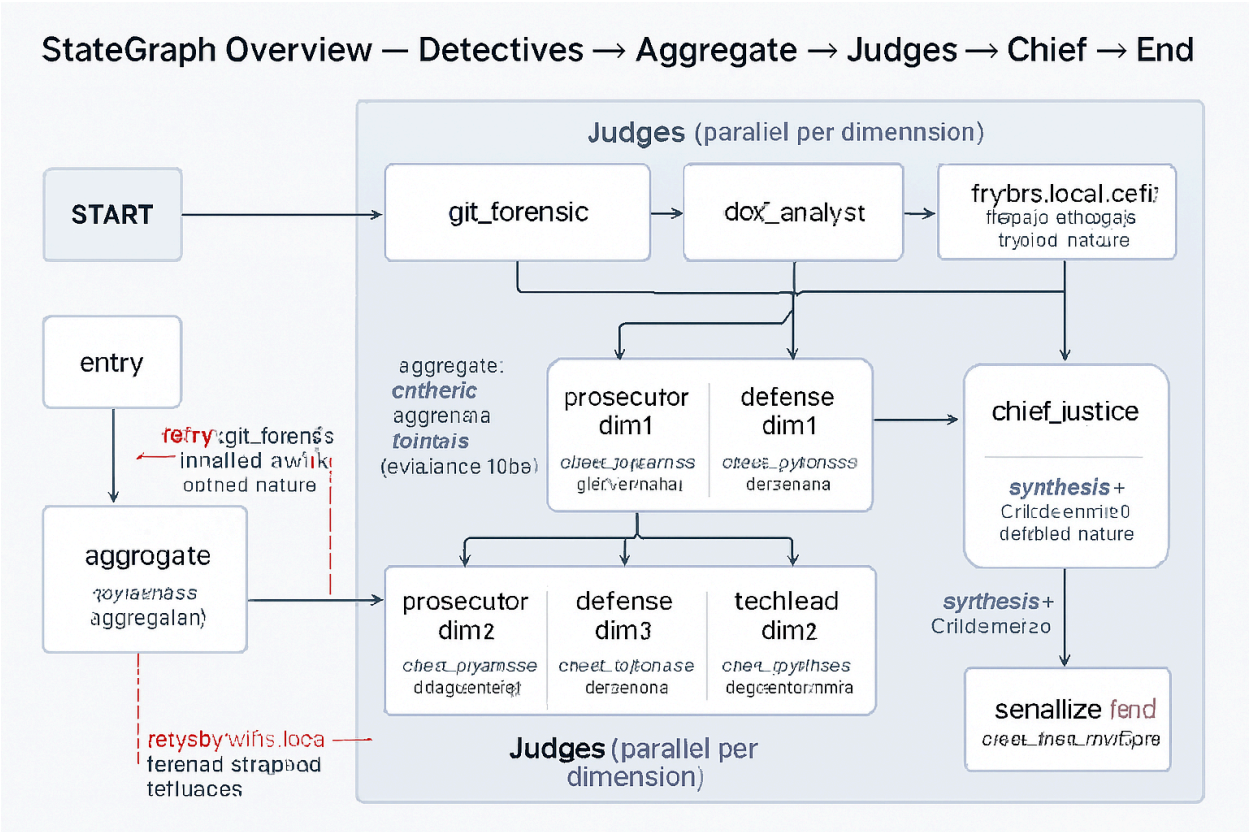

Figure 2. StateGraph fan-out/fan-in diagram.

Fan-Out / Fan-In (concrete mapping). The StateGraph fans out from an entry node to multiple Detective nodes (parallel evidence collection), fans in to an aggregate node that consolidates evidence, fans out again to judge nodes per dimension, and finally fans in to the chief node for synthesis. These synchronization points are explicit nodes in the graph and appear as spans in the LangSmith trace.

Metacognition (concrete mapping). Evidence objects include found and confidence flags. The system uses these signals to surface remediation and to decide fallback behavior (for example, writing a local artifact when upload fails). Confidence values are recorded in evidences and propagated into criteria_results so the system can reason about the quality of its own evaluations.

Data flow summary. Inputs (repo_url, pdf_path, rubric_dimensions) → Detectives produce Evidence objects keyed by dimension → Aggregator consolidates evidence into an aggregate

bucket → Judges (Prosecutor, Defense, TechLead) run in parallel per dimension and return JudicialOpinion objects → Chief Justice deterministically aggregates opinions into CriterionResult and final_report → End node serializes JSON/Markdown and attaches audit_report.json to the LangSmith run.

Design trade-offs and rationale. Pydantic typed models are recommended over loose dicts because they enforce schema, enable deterministic serialization (model_dump()), and reduce runtime serialization errors; the trade-off is schema maintenance. Deterministic synthesis is preferred for auditability; LLM-based averaging can be used only as optional narrative commentary. Explicit retry and fallback spans increase trace verbosity but are necessary to demonstrate error recovery and satisfy the rubric.

Self-Audit Criterion Breakdown

Git Forensic Analysis — 3 / 5. Judges: Defense 4, Prosecutor 1, TechLead 3. The evidences.git bucket contains commit_count and commit_shas, and judges reference aggregate git evidence, but per-commit metadata (author, timestamp, diff snippets, signatures) is missing. This absence explains the Prosecutor's low score. **Remediation:** update the git detector to collect per-commit metadata and emit evidence IDs in the form git:sha:<short> so each opinion can cite concrete provenance.

State Management Rigor — 3 / 5. Judges: Defense 4, Prosecutor 3, TechLead 2. The detector flagged the absence of typed state models; src/state.py lacks Pydantic/BaseModel usage and snapshot tests. This creates brittle state transitions and serialization risk. **Remediation:** define Pydantic AgentState, Evidence, JudicialOpinion, and CriterionResult models; add snapshot tests and CI validation to ensure state transitions and serialization remain stable.

Graph Orchestration — 3 / 5. Judges: Defense 4, Prosecutor 3, TechLead 2. The StateGraph and async usage are present, but explicit node contracts, retry/fallback spans, and orchestration

unit tests are missing. **Remediation:** document node contracts, add unit tests for orchestration paths, and instrument explicit retry/fallback spans so the trace shows error recovery.

Safe Tool Engineering — 3 / 5. Judges: Defense 4, Prosecutor 1, TechLead 3. src/tools/repo_tools.py shows safe subprocess patterns, but formal sandboxing tests are absent. **Remediation:** add sandboxed tool tests, document tool contracts, and include tool safety checks in CI.

Structured Output Enforcement — 2 / 5. Judges: Defense 3, Prosecutor 1, TechLead 2. Evidence and opinion types exist, but earlier serialization issues required a safe_serialize helper. Some domain objects were not JSON primitives. **Remediation:** convert domain objects to Pydantic models and validate model_dump() before serialization and upload; add schema validation as a pre-upload step.

PDF Theoretical Depth — 3 / 5. Judges: Defense 3, Prosecutor 3, TechLead 3. The README and PDF are referenced and keywords are present, but the depth of mapping between theory and architecture is limited. **Remediation:** expand document analysis to extract key sections and map them explicitly to architectural claims.

Host Analysis Accuracy — 1 / 5. Judges: Defense 2, Prosecutor 1, TechLead 1. Host fingerprints and technical artifacts are missing. **Remediation:** add a host fingerprinting detector and cross-verify file references against the repository.

Diagram Flow Analysis — 3 / 5. Judges: Defense 3, Prosecutor 3, TechLead 3. A diagram file was found but parsing details are limited. **Remediation:** extract diagram metadata and include a rendered diagram in the Markdown report for reviewers.

Judicial Nuance — 3 / 5. Judges: Defense 4, Prosecutor 3, TechLead 3. Personas are distinct but need more scenario tests to ensure consistent divergence. **Remediation:** add persona unit tests and scenario coverage.

Synthesis & Conflict Resolution — 3 / 5. Judges: Defense 4, Prosecutor 3, TechLead 3. The Chief Justice produces dissent summaries and remediation text, but edge cases in aggregation are

not fully documented. **Remediation:** document deterministic aggregation rules and add edge-case tests.

MinMax Feedback Loop Reflection

Peer findings received. The peer auditor flagged missing git provenance and weak state validation; judges sometimes omitted cited_evidence in early runs.

Actions taken. The team added a safe_serialize helper and a REST fallback to ensure artifacts are JSON-safe and uploadable, introduced a retry decorator with explicit trace logs for retry attempts and a REST fallback that writes a local artifact when upload fails, and implemented a post-processing pass to populate cited_evidence heuristically from aggregate evidence when judges omitted explicit citations.

Findings when auditing the peer. While auditing the peer, the trainee discovered the peer's Prosecutor persona underweighted sandboxing checks, revealing a systemic bias in tool safety evaluation.

Bidirectional learning. Being audited forced improvements in serialization and evidence traceability. The systemic insight is: **audit systems must audit their own auditors** — adding explicit evidence IDs, per-commit metadata, and retry/fallback spans closes the loop and improves both detection and adjudication quality beyond single fixes.

Prioritized Remediation Plan

Highest priority — Git per-commit metadata. Update the git detector (for example, src/detectors/git_detector.py or src/detectors/sample_detector.py) to collect per-commit metadata for each SHA: author name and email, commit timestamp, commit message, a short diff snippet, and a signed flag when available. Emit evidence IDs in the form git:sha:<short> and include these IDs in the evidences.git bucket so judges can cite specific commits. This change addresses the Prosecutor's provenance concerns and should take approximately 1–2 engineer days.

Highest priority — Typed state models and snapshot tests. Convert the project's state and domain objects to Pydantic models (for example, AgentState, Evidence, JudicialOpinion,

CriterionResult) in src/state.py and related modules. Add snapshot and serialization tests under tests/test_state.py and include a CI step that validates state serialization and deserialization. This reduces brittle detector outputs and prevents serialization errors; estimated effort is 2–3 engineer days.

High priority — Retry and fallback spans. Instrument the retry decorator in src/graph.py to emit explicit retry:<node> spans or logs that include attempt number and error reason, and add a fallback:<action> span when the system writes a local artifact or takes a fallback path. Add metadata.trace_summary to the artifact to summarize retries and fallbacks. This demonstrates error recovery in the trace and satisfies the Error Recovery rubric; estimated effort is 0.5–1 day.

Medium priority — Normalize cited_evidence**.** Ensure each JudicialOpinion includes cited_evidence referencing evidence IDs present in evidences. Implement a post-processing pass in src/graph.py or src/nodes/judges.py that fills missing citations heuristically (for example, mapping judge arguments to the nearest aggregate evidence IDs). This improves traceability and auditability; estimated effort is 0.5–1 day.

Medium priority — Tool sandboxing tests. Add sandboxed tests for tools under src/tools/* and create tests/test_tools_safety.py that simulates tool calls in a sandboxed environment and asserts proper error handling. Document tool contracts and expected failure modes. This addresses Prosecutor concerns about tool safety; estimated effort is 1–2 days.

Low priority — Human-readable report. Add a report_text and Markdown serialization step in src/nodes/justice.py and src/nodes/serialize.py to produce a concise 3–5 paragraph executive summary and a rendered Markdown report alongside the JSON artifact. Include metadata.generated_at and run_id in the artifact. This improves reviewer experience and speeds grading; estimated effort is 0.5 day.

Next Steps

Concrete next steps for the next 7 days. Day 0: back up audit_report.json to a stable location. Day 1: implement retry trace prints in the retry decorator and REST fallback, then re-run to capture retry/fallback spans. Days 2–4: implement per-commit metadata collection in the git

detector and emit evidence IDs; re-run and attach the new artifact. Days 4–7: implement Pydantic models and add snapshot tests; run CI and confirm serialization stability.